Performance & Capacicity of Erlang/GStreamer WebRTC Gateway

Zen Crypted X.422.2 RTP

July 11, 2026

Abstract

This paper presents a formal capacity and soundness analysis of a consolidated, monolithic WebRTC gateway architecture (`rtp-monolith`). By collapsing the signaling loop, process registry (`syn`), STUN/TURN traversal (`eturnal`), and database persistence (`mnesia`) into a single Erlang virtual machine (BEAM), and delegating video compositing directly to local GStreamer port processes, the system achieves significant resource savings over traditional multi-tier solutions. However, our mathematical models identify a severe asymmetry between the low-cost Erlang actor signaling layer and the compute-heavy software video compositing layer. We formulate these limits and propose SRE mitigations.

1 System Model & Parameter Definitions

We model the monolithic node as a closed queuing system handling concurrent signaling channels, relayed network packets, and active video compositing pipelines.

1.1 Memory Allocation Model

The total memory consumption M_{total} of the container is formulated as:

$$M_{\text{total}} = M_{\text{base}} + N \cdot M_{\text{conn}} + N_{\text{turn}} \cdot M_{\text{turn}} + R \cdot (M_{\text{room}} + M_{\text{gst}}) \quad (1)$$

Where:

- M_{base}: Base RAM overhead of the BEAM runtime and OS libraries (≈ 50 MB).

- N: Total concurrent client connection sessions.
- M_{conn}: Actor memory footprint per socket (server process + state) (≈ 2 KB).
- N_{turn}: Active relayed allocations managed by `eturnal` ($N_{\text{turn}} \leq N$).
- M_{turn}: Network allocation state allocation inside `eturnal` (≈ 10 KB).
- R: Number of active video conference rooms.
- M_{room}: Room coordinator GenServer process memory (≈ 5 KB).
- M_{gst}: RAM footprint of the GStreamer `compositor` OS process (≈ 250 MB).

1.2 CPU Utilization Model

The total CPU load C_{total} (expressed in Cores) is defined as:

$$C_{\text{total}} = C_{\text{beam}}(N) + C_{\text{turn}}(T) + R \cdot C_{\text{gst}}(K) \tag{2}$$

Where:

- $C_{\text{beam}}(N)$: CPU load of the BEAM signaling scheduler for N users (highly linear: 1.5×10^{-5} cores per active connection).
- $C_{\text{turn}}(T)$: CPU consumption of STUN/TURN packet forwarding, where T is the throughput in Mbps (Network I/O bound).
- $C_{\text{gst}}(K)$: CPU complexity of a GStreamer composition pipeline with K incoming streams.

2 Computational Complexity of GStreamer Compositing

The computational load of GStreamer compositing is a function of incoming resolutions, frame rates, color-space conversions (YUV to RGB and vice-versa), and final compression encoding:

$$C_{\text{gst}}(K) = \sum_{i=1}^{K} D_i(W_i, H_i, F_i) + \Phi\left(\sum_{i=1}^{K} W_i \cdot H_i\right) + E(W_{\text{out}}, H_{\text{out}}, F_{\text{out}}) \tag{3}$$

Where:

- D_i: Decoding complexity of stream i with width W_i, height H_i, and frame-rate F_i.
- Φ: Grid scaling and pixel blending transformation complexity.
- E: Encoder complexity (e.g., x264 software encoding) for output width W_{out} and height H_{out}.

2.1 Media Workload Parameters

To model the CPU and network footprint, the standard streaming codecs, resolutions, and target bitrates for both incoming client WebRTC streams and the output composited recording stream are defined in Table 1.

Table 1: Codec, Resolution, and Bitrate Parameters

Stream Type	Codec	Resolution
Incoming WebRTC (per client)	H.264 (Baseline Profile) or VP8	720p (1280×720) @ 30 fps
Output Recording (Composited)	H.264 (High Profile) via x264	1080p (1920×1080) @ 30 fps

For a standard court layout ($K = 4$ incoming streams at 720p @ 30fps composited into a single 1080p @ 30fps H.264 stream), the CPU core consumption without hardware acceleration (GPU/QuickSync) is calculated empirically as:

$$C_{\text{gst}}(4) \approx 0.15 \text{ (Decoders)} + 0.20 \text{ (Compositor)} + 0.65 \text{ (x264 software encoder)} = 1.00 \text{ Core} \tag{4}$$

3 Capacity Boundaries (4 Cores, 4 GB RAM)

Using the parameters defined above, we evaluate the maximum capacity limits of a single pod.

3.1 Scenario A: Idle Signaling & TURN Relay (No GStreamer Mixers)

Given: $R = 0$, $N = 10,000$, $N_{\text{turn}} = 2,000$ (relaying 2 Gbps aggregate traffic).

$$M_{\text{total}} = 50 \text{ MB} + (10,000 \cdot 2 \text{ KB}) + (2,000 \cdot 10 \text{ KB}) = 90 \text{ MB} \tag{5}$$

$$C_{\text{total}} = 0.15 \text{ Cores (Erlang)} + 0.80 \text{ Cores (TURN Relay)} = 0.95 \text{ Cores} \tag{6}$$

Evaluation: The node is highly stable. RAM utilization is at 2.25% and CPU is at 23.7%.

3.2 Scenario B: Active Recording & Compositing (Mixed Workload)

Given: $N = 1,000$, $N_{\text{turn}} = 100$, and R active mixing rooms (4 inputs each).

$$M_{\text{total}} = 50 \text{ MB} + 2 \text{ MB} + 1 \text{ MB} + R \cdot 250 \text{ MB} \leq 4,000 \text{ MB} \implies R \leq 15.78 \text{ Rooms}$$
$$C_{\text{total}} = 0.015 \text{ Cores} + 0.04 \text{ Cores} + R \cdot 1.00 \text{ Cores} \leq 4.00 \text{ Cores} \implies R \leq 3.94 \text{ Rooms}$$

Under standard CPU limits, the pod saturates at $R = 3$ active mixed rooms.

3.3 Target Scenario: Scaling to 50 Concurrent Active Video Rooms

To achieve the operational target of 50 concurrent active rooms, three architectural models are formulated:

1. **Vertical Monolith Node (Scale-Up)**: Host all 50 rooms inside a single monolithic VM/Pod container. Requires $M_{\text{total}} \approx 12.6$ GB RAM and $C_{\text{total}} \approx 50.1$ Cores. Requires a 52-Core / 16 GB RAM bare-metal server node.

2. **Horizontal Scaling (Scale-Out)**: Distribute 50 rooms across P isolated, independent Alpine Linux pods running on a 4-Core / 4 GB RAM boundary. Since $R_{\text{node}} = 3$, $P = \lceil 50/3 \rceil = 17$ Pod Replicas. Nodes operate as share-nothing instances without forming an Erlang cluster.

3. **Hardware-Accelerated Monolith (GPU-Offloaded)**: Offload H.264 software encoding to NVENC. The accelerated CPU complexity per room drops to $C_{\text{gst, gpu}}(4) \approx 0.40$ Cores. Total CPU for 50 rooms is $C_{\text{total}} \approx 20.05$ Cores. Requires a single 20-Core / 16 GB RAM GPU-enabled Kubernetes node with an NVIDIA T4/A10G card.

3.4 Extreme Scale Scenario: Scaling to 1000 Concurrent Active Video Rooms

At a massive scale of $R = 1,000$ active concurrent rooms (4,000 participants), resource calculations reveal severe hardware limits:

- **Memory Requirement**:

$$M_{\text{total}} = 50 \text{ MB} + (4,000 \cdot 2 \text{ KB}) + (400 \cdot 10 \text{ KB}) + (1,000 \cdot 5 \text{ KB}) + (1,000 \cdot 250 \text{ MB}) \approx 250.07 \text{ GB RA} \tag{7}$$

- **CPU Core Requirement (x264 Software Encoding)**:

$$C_{\text{total}} = 0.06 \text{ Cores} + 0.16 \text{ Cores} + 1,000 \cdot 1.00 \text{ Core} \approx 1,000.22 \text{ Cores} \tag{8}$$

- **CPU Core Requirement (GPU-Accelerated NVENC)**:

$$C_{\text{total, gpu}} = 0.06 \text{ Cores} + 0.16 \text{ Cores} + 1,000 \cdot 0.40 \text{ Cores} \approx 400.22 \text{ Cores} \tag{9}$$

4 Systems Soundness & Liveness Analysis

4.1 HPA Oscillation (Thundering Herd)

Configuring Kubernetes Horizontal Pod Autoscaling (HPA) against standard CPU limits (e.g., 80% target) creates severe scaling instabilities. When R goes from $1 \to 3$, CPU spikes from $26\% \to 80\%$, triggering replica creation. Because Erlang cluster state is localized on each pod, the new replica registers, but since incoming signaling sockets do not balance instantly, the CPU spike persists on the first node, prompting further HPA scale-ups until replica bounds are hit. The scaling algorithm must utilize custom metrics mapped to active GStreamer mixers (`active_gst_mixers`) instead of generic system CPU metrics.

4.2 Local Database Isolation (No Database Clustering)

Under the isolated pod model, Mnesia runs strictly as a local persistent storage database inside each pod's local directory (persisted via Kubernetes PVC on Alpine Linux). Because Erlang nodes do not form a cluster, all database transaction locking, schema updates, and state operations are restricted to the local node. This completely avoids split-brain partitions, node discovery synchronization issues, and cluster-wide database deadlocks during HPA scaling, ensuring high reliability and simplified operations.

4.3 Scale-Out Architecture Bottlenecks at 1000+ Rooms

When scaling the cluster horizontally to support $R = 1,000$ active rooms across ≈ 333 replicas of 4-Core pods, three critical infrastructure bottlenecks emerge:

1. **Erlang Distribution Overhead Bypassed**: By routing all room participants to the same pod via Ingress Sticky Session hashing, signaling is fully handled locally in-memory. Consequently, the Erlang distribution mechanism is completely disabled (the nodes run with `-no_distribution`), totally avoiding the quadratic $O(P^2)$ connection storm ($\approx 110,000$ links at $P = 333$ replicas) and ensuring that pods remain as isolated Alpine Linux containers.

2. **Mnesia Transaction Locking Isolated**: Since Mnesia runs as a purely local database on each isolated pod (with local folder persistence on Alpine Linux), transaction lock contention is strictly confined to the local node's processes. This eliminates distributed two-phase commit latency (RTT replication delays) and prevents cluster-wide Mnesia transaction deadlocks.

3. **Network Port & Bandwidth Saturation**: At an average stream bitrate of 1.5 Mbps, the aggregate network ingestion throughput reaches $4,000 \times 1.5$ Mbps $= 6.0$ Gbps. Under standard Kubernetes network overlays (e.g. Calico/Flannel), encapsulation overhead (VXLAN/Geneve) consumes substantial CPU, bottlenecking network card rings. SREs must deploy network interfaces using `hostNetwork` configurations with SRIOV-enabled interfaces to bypass container network encapsulation.

5 Conclusion

The proposed monolithic architecture is highly sound for large-scale signaling and STUN/TURN routing, easily supporting over $10,000$ concurrent user events. However, software-based video compositing introduces a severe compute limit, capping density at 3 rooms per pod on a 4-Core boundary. Transitioning to hardware-accelerated transcoding (NVENC/VAAPI) or partitioning media processing to dedicated GPU nodes is required to increase room density.